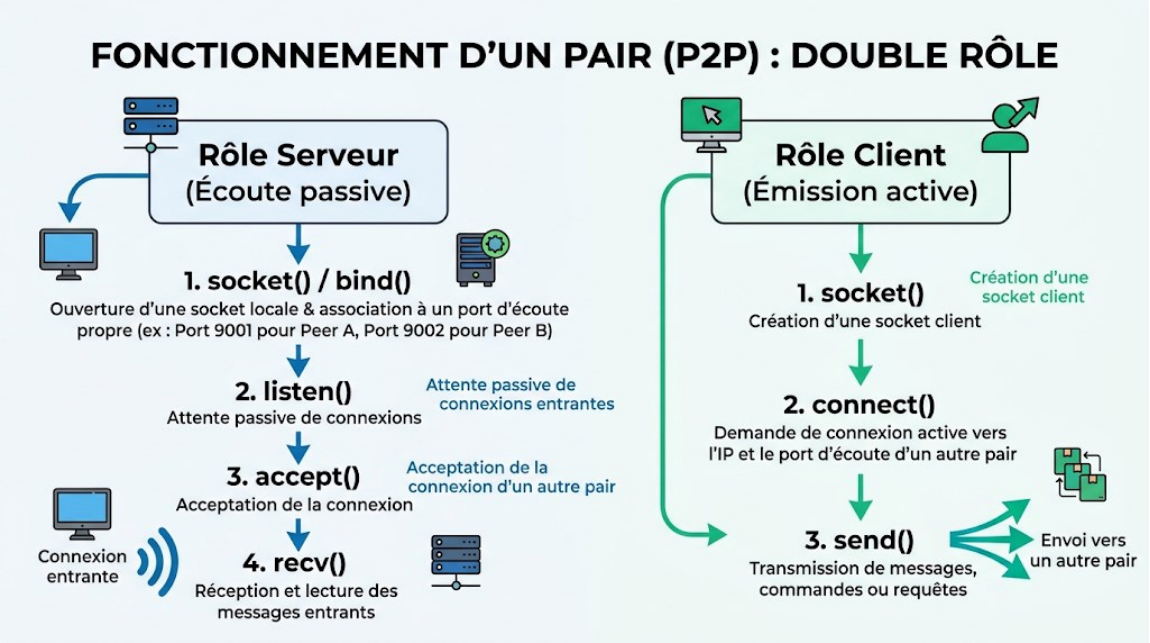
TP 2 : Communication Décentralisée Peer-to-Peer (P2P)

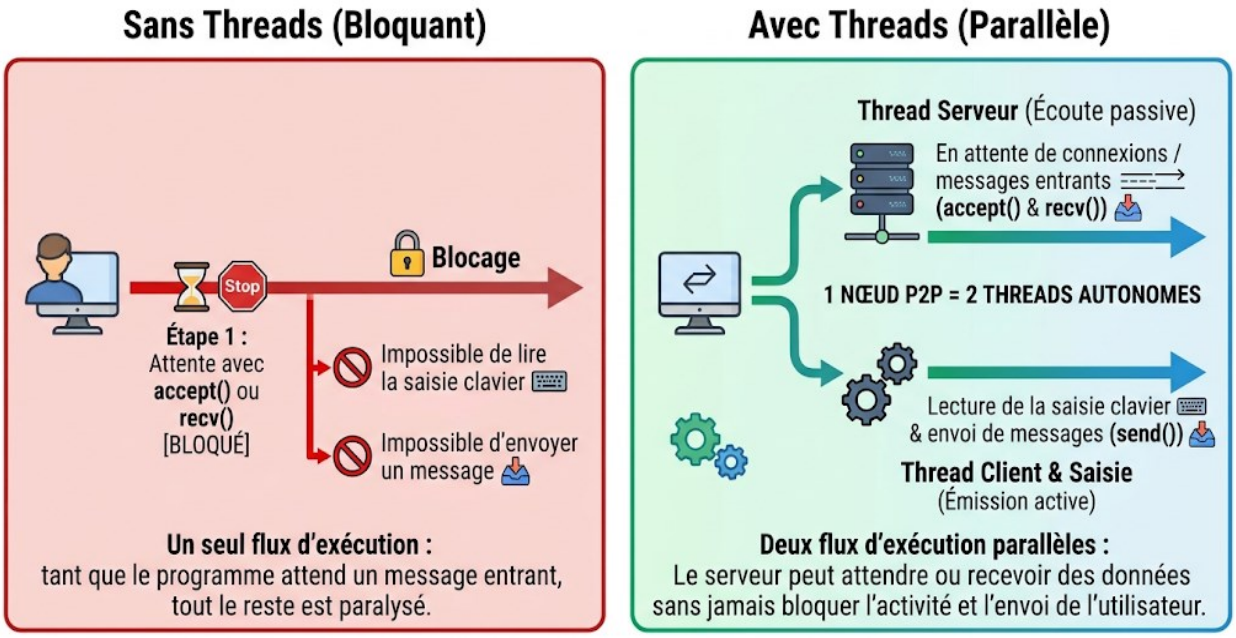
Étape 1 : Architecture et Déroulement de la Communication

Dans une architecture **Peer-to-Peer (Pair-à-Pair / P2P)**, cette asymétrie disparaît : chaque nœud du réseau (appelé **Peer** ou **Servent** pour *Server* + *Client*) possède simultanément les deux rôles.

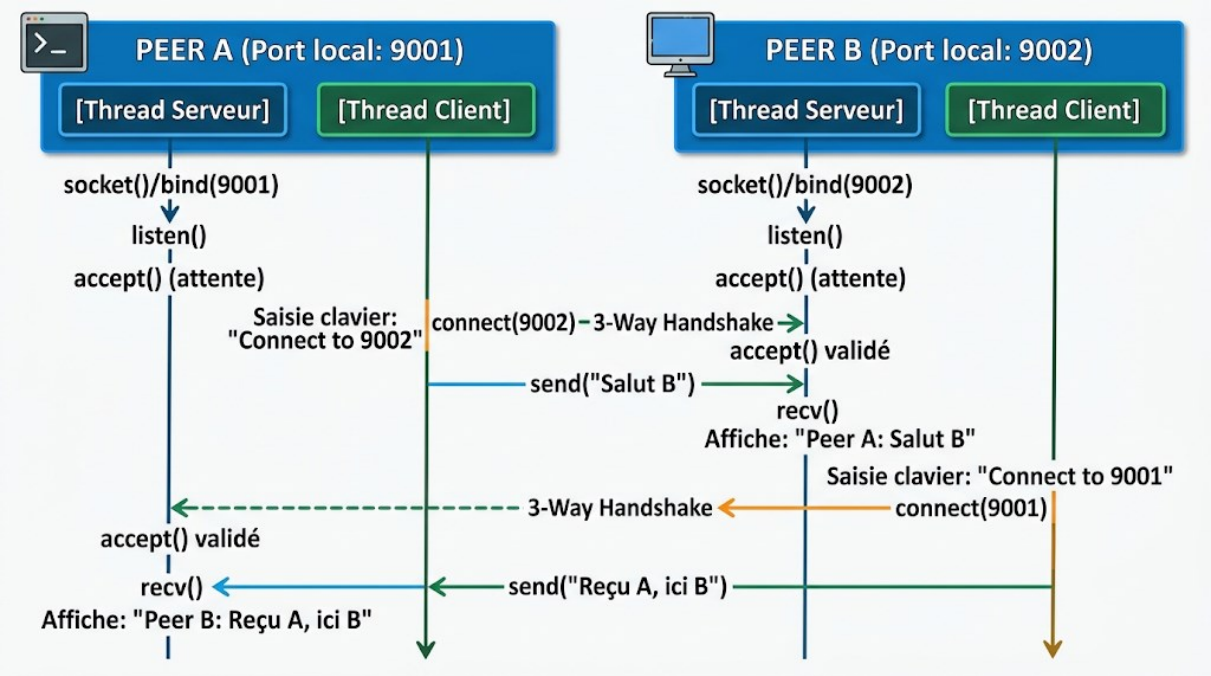
1. Caractéristiques Fondamentales d'un Nœud Peer



2. Le Défi de la Concurrence : Pourquoi les Threads sont indispensables ?



3. Chronogramme du Déroulement de la Communication entre 2 Pairs



Dans une architecture P2P, un pair doit **écouter** en continu les messages entrants tout en restant disponible pour **lire les saisies au clavier** de l'utilisateur.

- Sous Windows (windows.h), on délègue l'écoute passive à un thread d'arrière-plan grâce à l'API système CreateThread :

```
// Prototype obligatoire d'une fonction exécutée dans un thread Windows
DWORD WINAPI NomDeLaFonctionThread(LPVOID lpParam) {
    // Code exécuté en arrière-plan (ex: boucle accept() / recv())
    return 0;
}
```

- Pour lancer ce thread depuis le programme principal :

```
HANDLE hThread = CreateThread(
    NULL,                   // Sécurité par défaut
    0,                      // Taille de pile standard
    NomDeLaFonctionThread,  // Nom de la fonction à exécuter
    params,                 // Pointeur vers les arguments (ou NULL)
    0,                      // Démarrage immédiat
    NULL                    // Identifiant retourné (inutile ici)
);
```

Étape 2 : Code Source C d'un Nœud Pair (`peer.c`)

Dans ce programme, chaque instance exécutée reçoit en paramètre son **propre port d'écoute**. Le programme crée un thread secondaire d'écoute serveur en arrière-plan, tandis que le thread principal attend les commandes de l'utilisateur pour se connecter et envoyer des messages à un autre pair.

- Créez le fichier `peer.c` dans votre dossier de travail :

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <winsock2.h>
#include <ws2tcpip.h>
#include <windows.h>

#define BUFFER_SIZE 1024

// Structure pour passer les paramètres au thread serveur
typedef struct {
    int listen_port;
} ServerThreadParams;

// ================================================================================
// 1. THREAD SERVEUR (Tâche de fond) : Écoute passive des messages entrants
// ================================================================================
DWORD WINAPI ServerListenerThread(LPVOID lpParam) {
    ServerThreadParams *params = (ServerThreadParams *)lpParam;
    int listen_port = params->listen_port;
    free(params);

    SOCKET server_sock, client_sock;
    struct sockaddr_in server_addr, client_addr;
    int addr_len = sizeof(client_addr);
    char buffer[BUFFER_SIZE];

    // Création de la socket d'écoute
    server_sock = socket(AF_INET, SOCK_STREAM, 0);
    if (server_sock == INVALID_SOCKET) {
        printf("\n[THREAD SERVEUR] Erreur creation socket : %d\n", WSAGetLastError());
        return 1;
    }

    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(listen_port);

    // Liaison (bind) sur le port propre à ce pair
    if (bind(server_sock, (struct sockaddr *)&server_addr, sizeof(server_addr)) == SOCKET_ERROR)
{
        printf("\n[THREAD SERVEUR] Erreur bind sur le port %d : %d\n", listen_port,
WSAGetLastError());
        closesocket(server_sock);
        return 1;
    }

    // Mise en écoute (listen)
    if (listen(server_sock, 5) == SOCKET_ERROR) {
        printf("\n[THREAD SERVEUR] Erreur listen : %d\n", WSAGetLastError());
        closesocket(server_sock);
        return 1;
    }

    printf("[P2P] Port d'ecoute local %d active. Pret a recevoir des messages.\n", listen_port);
```

```
    // Boucle permanente de réception
    while (1) {
        client_sock = accept(server_sock, (struct sockaddr *)&client_addr, &addr_len);
        if (client_sock == INVALID_SOCKET) {
            continue;
        }

        // Réception du message transmis par un autre pair
        memset(buffer, 0, BUFFER_SIZE);
        int bytes_received = recv(client_sock, buffer, BUFFER_SIZE - 1, 0);
        if (bytes_received > 0) {
            buffer[bytes_received] = '\0';
            printf("\n--> [MESSAGE RECU] %s\n> ", buffer);
            fflush(stdout);
        }

        closesocket(client_sock);
    }

    closesocket(server_sock);
    return 0;
}

// ================================================================================
// 2. FONCTION CLIENT : Émission active vers un pair distant
// ================================================================================
void SendMessageToPeer(const char *target_ip, int target_port, const char *msg, int my_port) {
    SOCKET send_sock = socket(AF_INET, SOCK_STREAM, 0);
    if (send_sock == INVALID_SOCKET) {
        printf("[ERREUR CLIENT] Creation socket echouee.\n");
        return;
    }

    struct sockaddr_in target_addr;
    target_addr.sin_family = AF_INET;
    target_addr.sin_port = htons(target_port);
    target_addr.sin_addr.s_addr = inet_addr(target_ip);

    // Tentative de connexion vers le port d'écoute du pair distant
    if   (connect(send_sock,   (struct   sockaddr   *)&target_addr,   sizeof(target_addr))   ==
SOCKET_ERROR) {
        printf("[ERREUR CLIENT] Impossible de joindre le pair sur le port %d (est-il en ligne
?)\n", target_port);
        closesocket(send_sock);
        return;
    }

    // Construction d'une trame avec en-tête d'identification
    char formatted_payload[BUFFER_SIZE];
    snprintf(formatted_payload, sizeof(formatted_payload), "[De Peer %d] : %s", my_port, msg);

    // Envoi du message
    send(send_sock, formatted_payload, (int)strlen(formatted_payload), 0);
    printf("[CLIENT] Message transmis au pair distant %d.\n", target_port);

    closesocket(send_sock);
}

// ================================================================================
// 3. PROGRAMME PRINCIPAL : Démarrage et interface interactive
// ================================================================================
int main(int argc, char *argv[]) {
    if (argc < 2) {
```

```
        printf("Usage : %s <Port_Ecoute_Local>\n", argv[0]);
        printf("Exemple : %s 9001\n", argv[0]);
        return 1;
    }

    int my_listen_port = atoi(argv[1]);

    // Initialisation de la DLL Winsock
    WSADATA wsa;
    if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0) {
        printf("[ERREUR] WSAStartup : %d\n", WSAGetLastError());
        return 1;
    }

    // Préparation des paramètres du thread serveur
    ServerThreadParams *params = (ServerThreadParams *)malloc(sizeof(ServerThreadParams));
    params->listen_port = my_listen_port;

    // Lancement du thread serveur en arrière-plan
    HANDLE hThread = CreateThread(NULL, 0, ServerListenerThread, params, 0, NULL);
    if (hThread == NULL) {
        printf("[ERREUR] Echec creation du thread serveur.\n");
        WSACleanup();
        return 1;
    }

    // Petite temporisation pour laisser le serveur s'initialiser
    Sleep(500);

    printf("=================================================\n");
    printf("   NOEUD PEER ACTIF SUR LE PORT : %d\n", my_listen_port);
    printf("   Format d'envoi : <Port_Cible> <Message>\n");
    printf("   Exemple        : 9002 Bonjour camarade !\n");
    printf("   Quitter        : exit\n");
    printf("=================================================\n");

    char input[BUFFER_SIZE];
    while (1) {
        printf("> ");
        if (!fgets(input, sizeof(input), stdin)) break;

        // Suppression du saut de ligne final (\n)
        input[strcspn(input, "\r\n")] = 0;

        if (strcmp(input, "exit") == 0) {
            printf("[P2P] Arret du noeud.\n");
            break;
        }

        if (strlen(input) == 0) continue;

        // Découpage : Extraction du port cible et du corps du message
        char *target_port_str = strtok(input, " ");
        char *message_body = strtok(NULL, "");

        if (!target_port_str || !message_body) {
            printf("[Syntaxe invalide] Format attendu : <Port_Cible> <Message>\n");
            continue;
        }

        int target_port = atoi(target_port_str);
        SendMessageToPeer("127.0.0.1", target_port, message_body, my_listen_port);
    }
```

```
    // Fermeture propre
    CloseHandle(hThread);
    WSACleanup();
    return 0;
}
```

Étape 3 : Compilation et Simulation d'un Réseau P2P dans VS Code

- Ouvrez le terminal de VS Code (Command Prompt) et compilez le programme unique :

```
gcc peer.c -o peer.exe -lws2_32
```

- Un seul exécutable `peer.exe` suffit, car le même code sert à tous les nœuds du réseau.
- Dans le Terminal 1 (Nœud A) :

```
.\peer.exe 9001
```

- Dans le Terminal 2 (Nœud B) :

```
.\peer.exe 9002
```

- Dans le Terminal 3 (Nœud C) :

```
.\peer.exe 9003
```

- Depuis le Terminal 1 (Peer 9001), envoyez un message au pair 9002 :

```
> 9002 Salut B, as-tu recu les releves de temperature ?
```

Étape 4 : Exercice Pratique : Gestion d'une table de pairs voisins et commande `PEERS`

Objectif : Éviter de taper manuellement le port cible à chaque message et maintenir dynamiquement une liste des pairs connus**.**

- Définir un tableau statique stockant les ports des voisins connus :

```
(ex: int known_peers[10]; int peer_count = 0;)
```

- Permettre à l'utilisateur d'ajouter un voisin via la console grâce à la commande :

```
ADD <port>
```

- Ajouter la commande `PEERS` qui affiche la liste des ports des pairs actuellement enregistrés.
- Scénario de Test en Séance :
    - Taper `PEERS` : le programme affiche que la table est vide.
    - Taper `ADD 9001` : le programme refuse (`Impossible de s'ajouter soi-meme`).
    - Taper `ADD 9002` puis `ADD 9003` : le programme confirme les deux ajouts.
    - Taper à nouveau `ADD 9002` : le programme refuse car le pair est déjà présent.
    - Taper `PEERS` : les ports `9002` et `9003` apparaissent dans la liste formatée.